CYPRUS INTERNATIONAL UNIVERSITY

CMPE411
Systems Programming

CHAPTER 5 – Sockets and Network Programming

Assoc.Prof.Dr. Tamer TULGAR

Spring 2019-2020

Client/Server Communication

The Client/Server model is an application development architecture designed to separate the presentation of data from its internal processing and storage.

The client places a request or order to the server.

The server processes the request of the client

Bob

[Client]

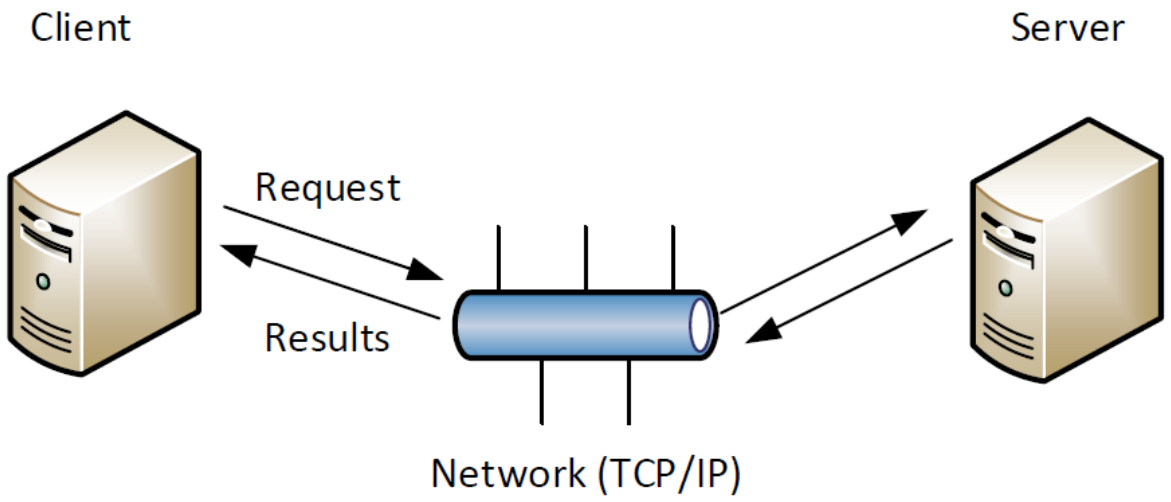
The communication between the client and the server is usually through a network.

Pizza Store

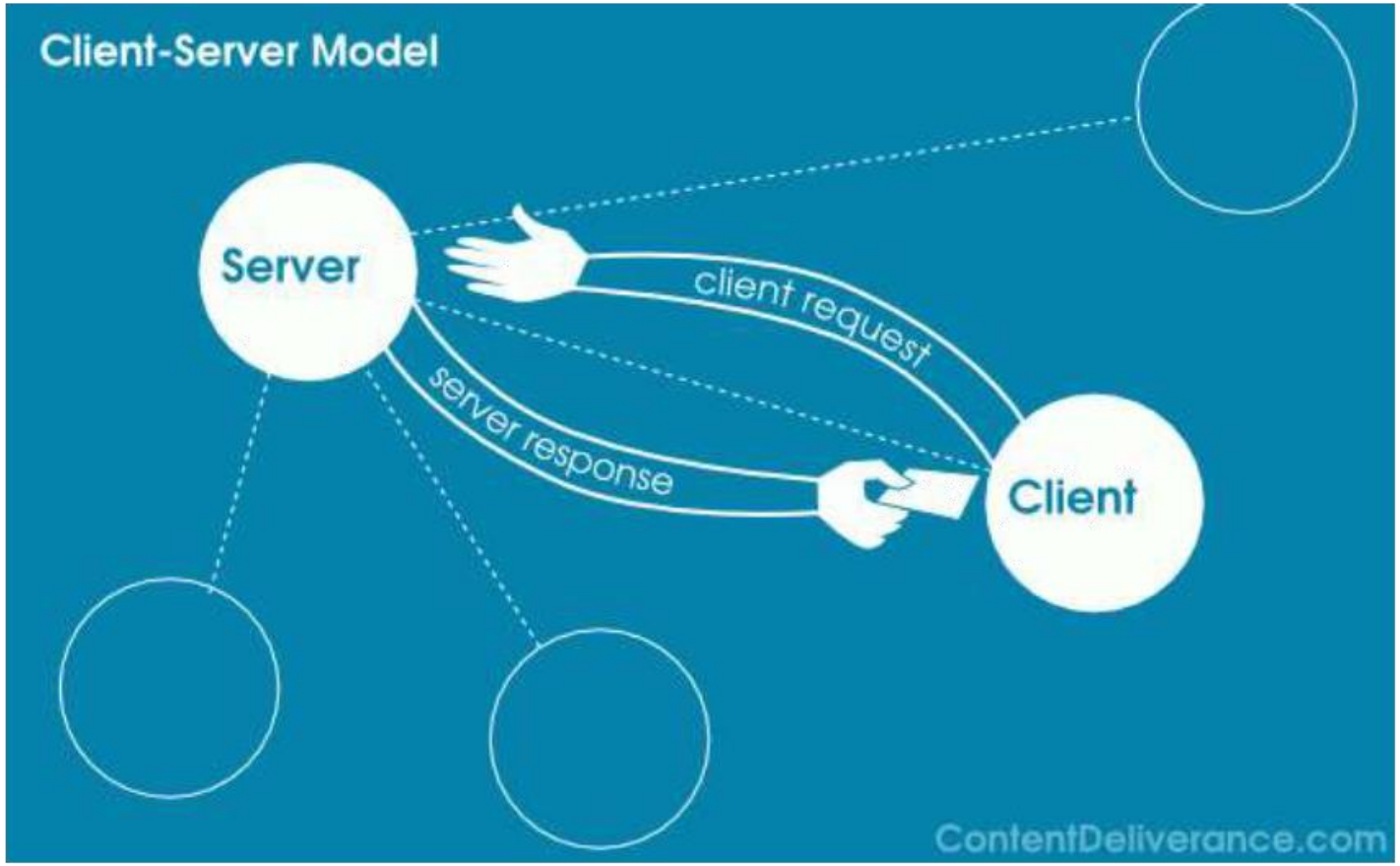
[Server]

Client/Server Communication

- The simplest level of network communication
- Consists of a server, client, and a network for communication
- **Client**: a computer running **"a program that makes a request for services"**
- **Server**: a computer running **"a program that offers requested services"**
  e.g., processing database queries from one or more clients

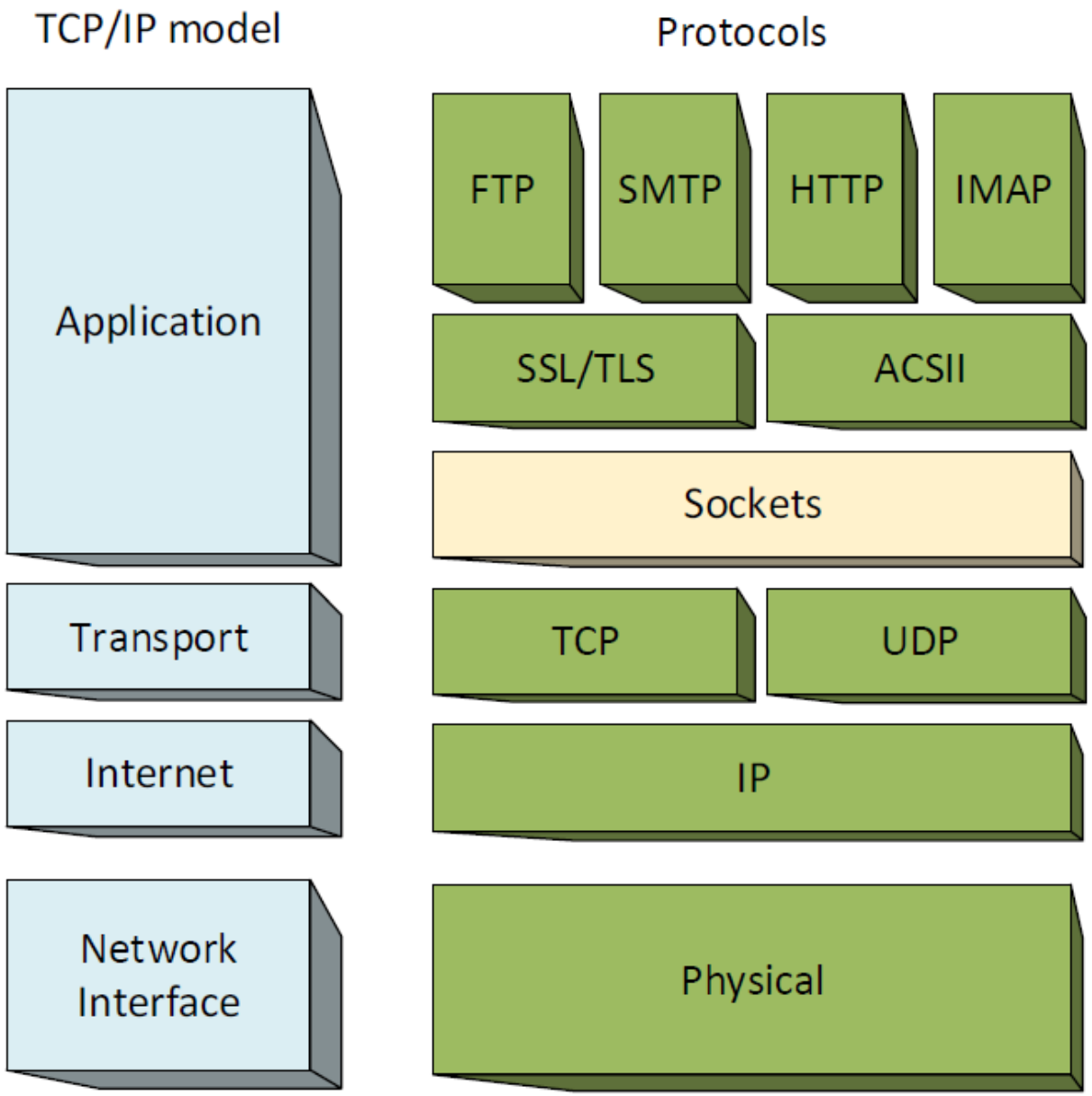


Client/Server Communication



The server and the client are not necessarily hardware components. They can be programs working on the same machine or on different machines.

TCP/IP Model - Application Layer : Sockets



TCP/IP Model - Application Layer : Sockets

- Sockets are the end points of connections between two hosts and can be used to send and receive data
- The lowest level before Transport layer
- Provides an interface for programming network communication.
- Similar to performing file I/O, e.g., socket handle is treated like a file handle.
- Independent of a programming language

> Sockets are used to handle the communication links between applications over the network. Further communication between the client and the server is through the socket.
>
> *-- Source: NIIT Courseware Q7M1*

TCP/IP Model - Application Layer : Sockets

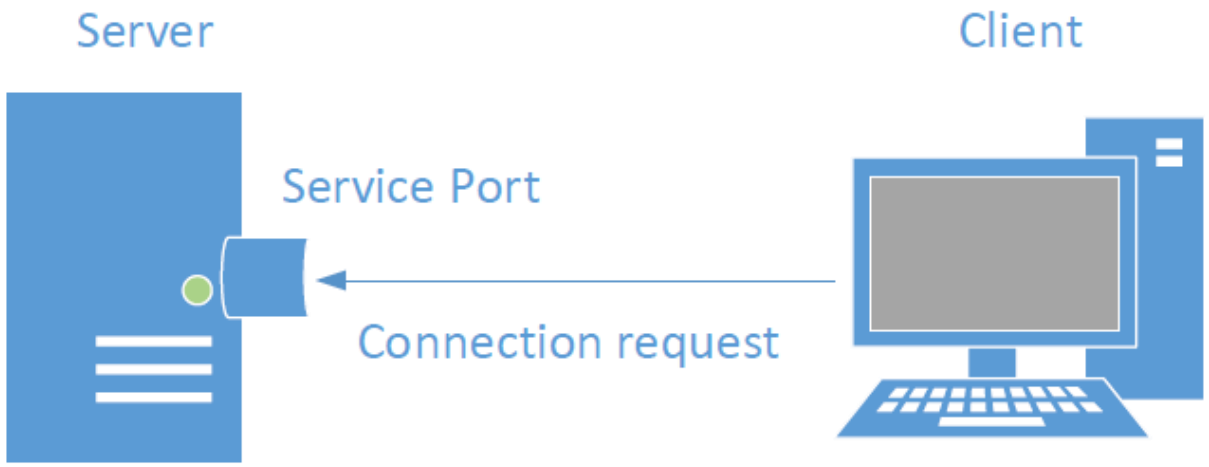
A socket consists of:

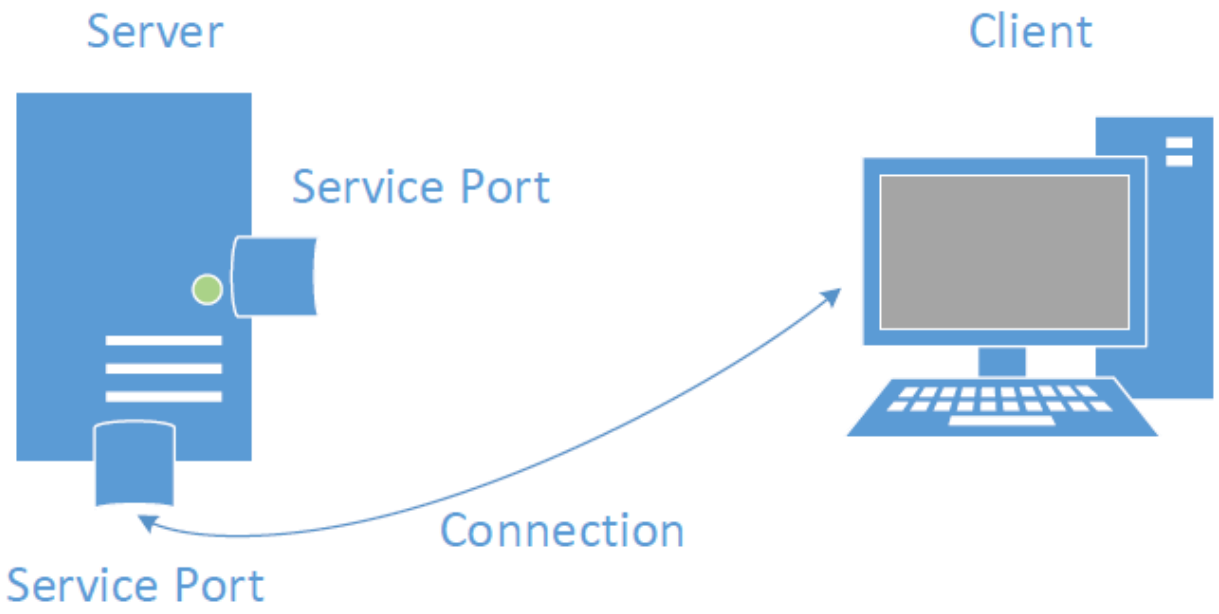
- Local socket address: Local IP address and service port number
- Remote socket address: Only for established TCP sockets
- Protocol: A transport protocol, e.g., TCP or UDP.
- A socket address is the combination of an IP address (phone number) and service port number (extension).
- A socket API is an application programming interface (API), usually provided by the operating system.

Service Ports

- Computers often communicate and provide more than one type of service or to talk to multiple hosts/computers at a time
- The port number is necessary to distinguish different server applications running on the same host.
- Each service offered by a computer is identified by a port number
- Represented as a positive (16-bit) integer value (integer less than or equal to 65565)
- Ports 1 through 1023 are reserved for administrative purposes (e.g. 21 for FTP, 23 for Telnet, 25 for e-mail, and 80 for HTTP).
- User-level process/services use port number value > 1024

Service Ports



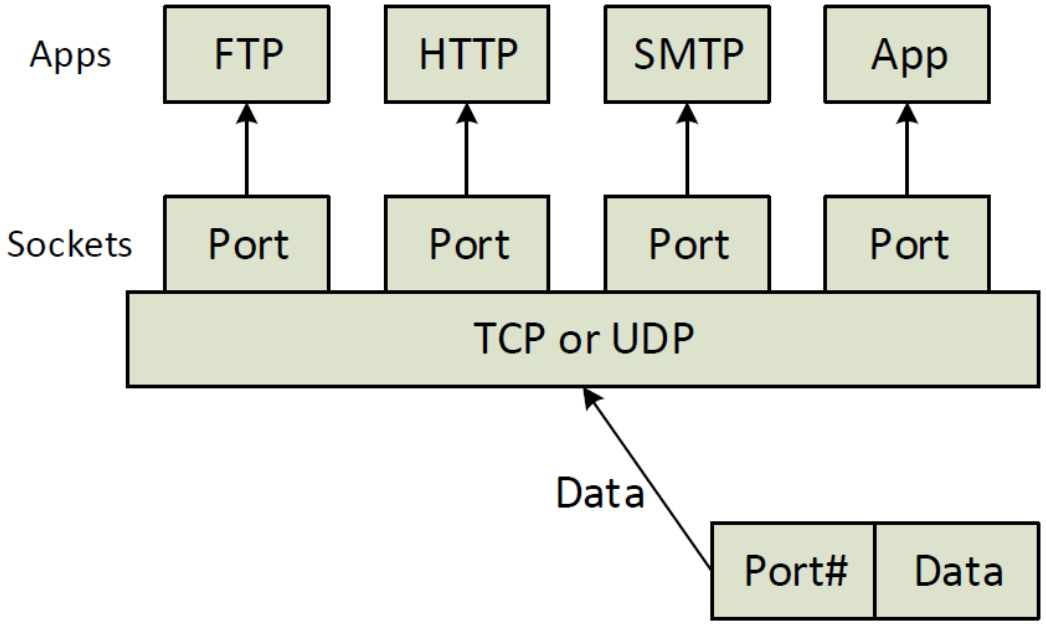


How to establish a connection

- Step 1: The server listens to the socket for a client to make a connection request
- Step 2: the server accepts the connection if everything goes well
- Step 3: the server gets a new socket bound to a different port
- Go to Step 1

How to establish a connection

- Client is connected to a server at the physical address identied by a IP address.
- Connection binds a port number to a socket number.
- Both TCP and UDP protocols use ports to map incoming data to a particular process running on a computer.



Sockets in JAVA (java.net package)

- The java.net package of the Java programming language contains classes and interfaces that provide support for networking.
- Networking classes contain methods to perform tasks, such as opening and closing a connection to remote machine, sending and receiving data packets and accessing resources on the Web.

Class	Description
DatagramPacket	This class represents a datagram packet.
DatagramSocket	This class represents a socket for sending and receiving datagram packets.
InetAddress	This class represents an Internet Protocol (IP) address.
MulticastSocket	The multicast datagram socket class is useful for sending and receiving IP multicast packets.
ServerSocket	This class implements server sockets.
Socket	This class implements client sockets (also called just "sockets").
URL	A pointer to a "resource" on the World Wide Web.
URLConnection	The superclass of all classes that represent a communications link between an application and a URL.

Class InetAddress

```
public boolean equals(Object obj);

public byte[] getAddress();
public static InetAddress[] getAllByName(String host);
public static InetAddress getByName(String host);
public String getHostName();
public static InetAddress getLocalHost();

public int hashCode();
public String toString();
```

This class represents an Internet Protocol (IP) address.

Applications should use the methods `getLocalHost()`, `getByName()`, or `getAllByName()` to create a new `InetAddress` instance.

Class InetAddress

```
import java.net.*;

public class HostInfo
{
  public static void main( String args[] )
  {
       InetAddress ipAddr;

      try
      {
            ipAddr = InetAddress.getLocalHost();
            System.out.println( "This is " + ipAddr );
      }
       catch ( UnknownHostException ex )
      {
            System.out.println( "Unknown host" );
      }
}
```

Java.net for Client/Server Applications

- Java provides a set of classes in java.net package, to enable the rapid development of network applications.
- Essentially provides classes, interfaces, and exceptions to simplify the complexity involved in developing client and server programs
- Two key classes for creating server and client programs
  - **ServerSocket**
  - **Socket**

Class ServerSocket

- Represents a TCP/IP server socket object that receives connection requests from the clients.

- A **`ServerSocket`** waits for requests to come in over the network. It performs some operation based on that request, and then possibly returns a result to the requester.

- The actual work of the **`ServerSocket`** is performed by an instance of the `SocketImpl` class.

- The abstract class `SocketImpl` is a common superclass of all classes that actually implement sockets. It is used to create both client and server sockets.

- A *plain* socket implements the SocketImpl methods exactly as described, without attempting to go through a firewall or proxy.

Class Socket

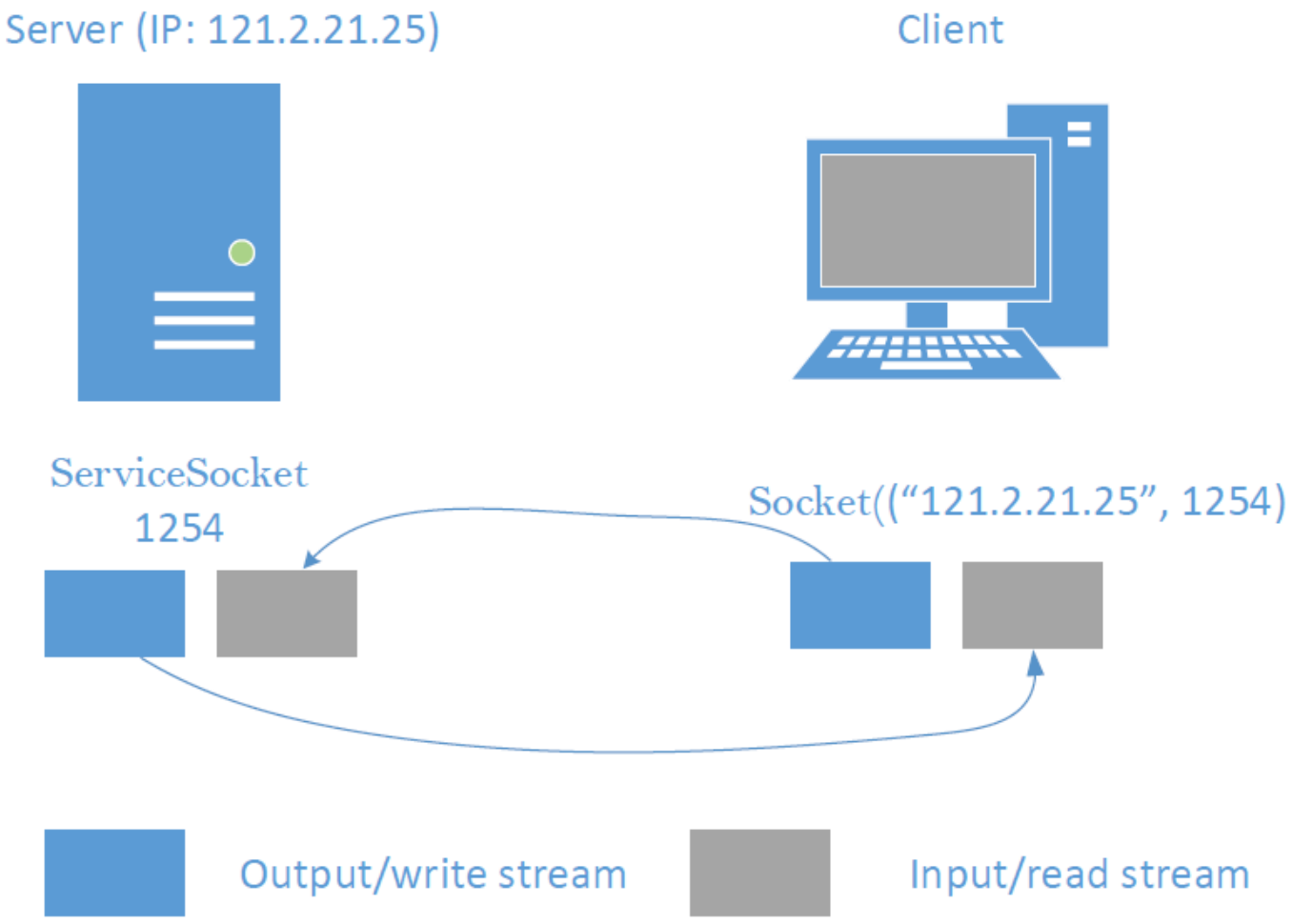
- This class implements client sockets (also called just *sockets*). A socket is a end point for communication between two machines.

- The actual work of the socket is performed by an instance of the `SocketImpl` class.

- It is possible to modify some TCP parameters:
  - `SO_LINGER`
  - `SO_TIMEOUT`
  - `TCP_NODELAY`
  - Enable/disable `TCP_NODELAY` (disable/enable Nagle's algorithm).

Socket-based client/server programming

- Server: **`ServerSocket`** creates a new service socket (port)
- Client: **`Socket`** establishes connection with service socket
- Both: exchange data using input and output streams



Socket-based client/server programming

Steps to create a simple server program:

- Step 1: Open the Server Socket:
  `ServerSocket server = new ServerSocket( PORT );`
- Step 2: Wait for the Client Request:
  `Socket client = server.accept();`
- Step 3: Create I/O streams for communicating to the client
  `DataInputStream is =`
  `new DataInputStream(client.getInputStream());`
  `DataOutputStream os =`
  `new DataOutputStream(client.getOutputStream());`
- Step 4: Perform communication with client
  - Receive from client: `String line = is.readLine();`
  - Send to client: `os.writeBytes("Hello!");`
- Step 5: Close the stream `is.close();` `os.close();`
- Step 6: Close the socket: `client.close();`

Socket-based client/server programming

CLIENT:

- Step 1: Open a socket.
- Step 2: Open an input stream and output stream to the socket.
- Step 3: Read from and write to the stream according to the server's protocol.
- Step 4: Close the streams.
- Step 5: Close the socket.

Let Us Remember Streams !!! (FILE I/O CHAPTER)

PrintWriter vs DataOutputStream

- **`PrintWriter`** : Prints formatted representations of objects to a text-output stream.
- **`DataOutputStream`** : A data output stream lets an application write primitive Java data types to an output stream in a portable way
- **PrintWriter** inherited from Writer class, while DataOutputStream inherited from OutputStream .
- OutputStream (therefore DataOutputStream) is designed for binary data, while Writer (therefore PrintWriter) is designed for text data

Let Us Remember Streams !!! (FILE I/O CHAPTER)

- Please Pay attention that the previous slide's reminder also applies for DataInputStream and BufferedReader.

- **`DataInputStream`** -> Byte Stream

- **`BufferedReader`** -> Character Stream

A First Example : Echo Service Server

```java
import java.io.*;
import java.net.*;

public class EchoServer
{
    public static void main(String[] args)
    {
        try
        {
            ServerSocket s = new ServerSocket( port: 8008);

            while (true)
            {
                Socket incoming = s.accept();

                BufferedReader in = new BufferedReader(new InputStreamReader(incoming.getInputStream()));

                PrintWriter out = new PrintWriter(new OutputStreamWriter(incoming.getOutputStream()));

                out.println("Hello... This is the Java EchoServer.");
                out.println("Enter BYE to exit.");
                out.flush();

                while (true)
                {
                    String str = in.readLine();
                    if (str == null)
                        break; // client closed connection

                    System.out.println(incoming.getInetAddress() + "  says-> " + str);

                    out.println("Echo: " + str);
                    out.flush();

                    if (str.trim().equals("BYE"))
                        break;
                }

                in.close();
                out.close();
                incoming.close();
            }
        } catch (Exception e) {}
    }
}
```

The echo server simply waits for a str as a request from the client and echoes it back to the client

A First Example : Echo Service Client

```java
import java.io.*;
import java.net.*;

public class EchoClient
{
    public static void main(String[] args)
    {
        try
        {
            String host = args.length > 0 ? args[0] : "localhost";

            Socket socket = new Socket(host, port: 8008);

            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(new OutputStreamWriter(socket.getOutputStream()));

            // send data to the server !
            for (int i = 1; i <= 10; i++)
            {
                System.out.println("Sending: line " + i);
                out.println("line " + i);
                out.flush();
            }

            out.println("BYE");
            out.flush();

            // receive data from the server

            while (true)
            {
                String str = in.readLine();
                if (str == null)
                    break;

                System.out.println(str);
            }

            in.close();
            out.close();
            socket.close();
        } catch (Exception e) {}
    }
}
```

The echo client sends a str as a request to the server and displays the incoming response from the server.

A First Example : Echo Service Sample Run

```
Command Prompt - java EchoServer

Z:\>java EchoServer
/192.168.2.4  says-> test 123
/192.168.2.4  says-> Hello from CMPE411
/192.168.2.4  says-> Sys Prog
/192.168.2.4  says-> BYE
```

telnet 192.168.2.7 8008

```
Command Prompt

Hello... This is the Java EchoServer.
Enter BYE to exit.
test 123
Echo: test 123
Hello from CMPE411
Echo: Hello from CMPE411
Sys Prog
Echo: Sys Prog
BYE
Echo: BYE


Connection to host lost.
```

```
Command Prompt - java EchoServer

Z:\>java EchoServer
/192.168.2.4  says-> test 123
/192.168.2.4  says-> Hello from CMPE411
/192.168.2.4  says-> Sys Prog
/192.168.2.4  says-> BYE
/192.168.2.4  says-> line 1
/192.168.2.4  says-> line 2
/192.168.2.4  says-> line 3
/192.168.2.4  says-> line 4
/192.168.2.4  says-> line 5
/192.168.2.4  says-> line 6
/192.168.2.4  says-> line 7
/192.168.2.4  says-> line 8
/192.168.2.4  says-> line 9
/192.168.2.4  says-> line 10
/192.168.2.4  says-> BYE
```

```
D:\Profile\Desktop\New folder>java EchoClient 192.168.2.7
Sending: line 1
Sending: line 2
Sending: line 3
Sending: line 4
Sending: line 5
Sending: line 6
Sending: line 7
Sending: line 8
Sending: line 9
Sending: line 10
Hello... This is the Java EchoServer.
Enter BYE to exit.
Echo: line 1
Echo: line 2
Echo: line 3
Echo: line 4
Echo: line 5
Echo: line 6
Echo: line 7
Echo: line 8
Echo: line 9
Echo: line 10
Echo: BYE
```

A Second Example : Echo Service with many clients

```java
import java.net.*;
import java.io.*;

class ClientHandler implements Runnable
{
    protected Socket incoming;

    public ClientHandler(Socket incoming) { this.incoming = incoming; }

    public void run()
    {
        try
        {
                BufferedReader in = new BufferedReader(new InputStreamReader(incoming.getInputStream()));
                PrintWriter out = new PrintWriter(new OutputStreamWriter(incoming.getOutputStream()));

                out.println("Hello! ...");
                out.println("Enter BYE to exit.");
                out.flush();

                while (true)
                {
                    String str = in.readLine();
                    if (str == null)
                        break;

                    System.out.println(incoming.getInetAddress() + "  says-> " + str);

                    out.println("Echo: " + str);
                    out.flush();

                    if (str.trim().equals("BYE"))
                        break;
                }

                in.close();
                out.close();
                incoming.close();

        } catch (Exception e) {}
    }
}
```

In this case, the server should handle each client request using a separate Thread !!!

A Second Example : Echo Service with many clients

```
public class MultiEchoServer
{
    public static void main(String[] args)
    {
        try
        {
            ServerSocket s = new ServerSocket( port: 8009);
            while (true)
            {
                Socket incoming = s.accept();
                new Thread(new ClientHandler(incoming)).start();
            }
        } catch (Exception e) {}
    }
}
```

In this case, the server should handle each client request using a separate Thread !!!

Using UDP (User Datagram Protocol)

- UDP is a fast, connectionless, and unreliable protocol.
  - no "connection" between client and server
  - no handshaking
  - sender explicitly attaches IP address and port of destination to each packet
  - server must extract IP address, port of sender from received packet
- transmitted data may be received out of order, or lost

UDP provides unreliable transfer of groups of bytes ("datagrams") between client and server

- UDP sockets use UDP protocol for communication over a network.

- The java.net package provides the following two classes that enable you to implement UDP sockets in a Java application:
  - **`DatagramPacket class`**
  - **`DatagramSocket class`**

Using UDP (User Datagram Protocol)

- The **`DatagramPacket`** object is a data container that consists of datagram packets that are sent or received over the network.
- The constructor used to initialize DatagramPacket objects are:
  - `public DatagramPacket( byte[] buffer, int buffer_length)`
  - `public DatagramPacket( byte[] buffer, int buffer_length, InetAddress address, int port)`

Using UDP (User Datagram Protocol)

- **`DatagramSocket`** class encapsulates the functionality to handle DatagramPacket objects.

- The constructors used to initialize DatagramSocket object are:
  - `public DatagramSocket()`
  - `public DatagramSocket(int port)`
  - `public DatagramSocket(int port, InetAddress address)`

Using UDP (User Datagram Protocol)

```
import java.net.*;
public class DataSender
{
    public DataSender()
    {
        try
        {
            DatagramSocket ds = new DatagramSocket();
            String str = "Hello World!";
            InetAddress ip = InetAddress.getByName("127.0.0.1");

            DatagramPacket dp = new DatagramPacket(str.getBytes(), str.length(), ip,  port: 3000);
            ds.send(dp);
            ds.close();
        }
        catch(Exception e) {System.err.println(e); }
    }
}
```

Using UDP (User Datagram Protocol)

```java
import java.net.*;
public class DataReceiver
{
    public DataReceiver()
    {
        try
        {
            DatagramSocket ds = new DatagramSocket( port: 3000);

            byte buf[] = new byte[1024];

            DatagramPacket dp = new DatagramPacket(buf,  length: 1024);

            ds.receive(dp);

            String str = new String(dp.getData(),  offset: 0, dp.getLength());

            System.out.println(str);

            ds.close();
        }
        catch(Exception e) {System.err.println(e); }
    }
}
```

Using UDP (User Datagram Protocol)

- **Homework:**

Please recode the EchoServer samples given for TCP Sockets employing UDP.